

# Django Templates Notes

Topic: Control Flow & Tags (If, For, With, Cycle, etc.) | By: Mohit Decodes

## 1. If-Else Statement

Django templates mein conditions (logic) lagane ke liye **if** aur **else** ka use kiya jata hai. Ye bilkul Python ke if-else jaisa hi kaam karta hai.

```
{% if condition %} <!-- Code to run if true --> {% else %} <!--  
Code to run if false --> {% endif %}
```

**Example:** Agar aapko check karna hai ki koi blog 'Featured' hai ya nahi, toh aap condition laga sakte hain. Agar ``is_feature`` True hai toh blog dikhega, warna ``else`` wala part run hoga.

## 2. For Loop & Empty Tag

List ya dictionary ke items ko ek-ek karke frontend par dikhane ke liye **for loop** ka istemaal hota hai.

```
{% for item in my_list %} <p>{{ item.name }}</p> {% empty %} <p>No  
blogs available!</p> {% endfor %}
```

- **Empty Tag:** Agar backend se aane wali list khali (empty) hai, toh loop break nahi hota balki seedha `{% empty %}` ke andar ka code chal jata hai.
- **Forloop Counter:** Loop ke andar elements ko ginne ke liye (1, 2, 3...) `{{ forloop.counter }}` ka use karte hain. (Yeh 1 se start hota hai, 0 se nahi).

## 3. With Tag

**With tag** ka use kisi complex calculation ya value ko ek temporary variable mein store karne ke liye kiya jata hai, taaki use baar-baar nikalna na pade. Isse code clean rehta hai aur performance bhi achhi hoti hai.

```
{% with total_comments=blogs.comments|length %} Total Comments: {{  
total_comments }} {% endwith %}
```

## 4. Cycle Tag

Values ko bari-bari se (alternate) print karne ke liye **cycle** tag use hota hai. Jaise agar

## 5. Firstof Tag

Yeh tag un variables ki list check karta hai jo aapne diye hain, aur usme se **pehli true/available value** ko print kar deta hai.

```
{% firstof blog.author blog.editor 'Anonymous User' %}
```

**Use case:** Agar kisi blog ka Author nahi likha hai, toh wo Editor ka naam dikha dega, aur agar Editor bhi nahi hai toh aakhir mein 'Anonymous User' print kar dega.

## 6. Now Tag

Current (aaj ki) date aur time ko format karke dikhane ke liye **now** tag ka use karte hain. Yeh server ke real-time ko dikhata hai.

```
{% now "jS F Y H:i" %} <!-- Output: 24th September 2026 11:30 -->
```

## 7. Verbatim Tag

Django template engine `{{ }}` aur `{% %}` ko apne variables ya tags samajhta hai. Agar aap Vue.js ya React use kar rahe hain jahan bhi yahi curly braces use hote hain, toh Django error de sakta hai.

Isse bachne ke liye hum **verbatim** tag ka use karte hain. Is tag ke andar ka code Django parse nahi karta, balki use as-it-is plain text ki tarah bhej deta hai.

```
{% verbatim %} {{ vue_variable_name }} {% endverbatim %}
```

## 8. Autoescape & Safe Filter (Security)

Django security ke mamle mein kaafi strong hai aur yeh automatically XSS (Cross-Site Scripting) attacks se bachata hai.

- **Autoescape:** Django by-default kisi bhi HTML tag ko text (string) mein convert kar deta hai (escape kar deta hai) taaki koi hacker input box ke zariye malicious scripts na chala sake.
- **Safe Filter (`|safe`):** Agar aap backend se khud jaan-bujhkar safe HTML bhej rahe hain (jaise `<b>Hello</b>`) aur chahte hain ki yeh browser par as a bold text render ho, toh aap `|safe` filter laga sakte hain:  
`{{ blog.content|safe }}`
- **Autoescape Off:** Aap chahein toh `{% autoescape off %}` tag ka use karke pure block ke liye escaping band kar sakte hain, par aisa karna risky hota hai agar data